Lab 5: Actuator Control and Safe Power Systems

Servo Position or DC-Motor Driver Track

ENGR 1105 | Fall 2026

Name	**Partner**
Date	**Section**

Learning Objectives

By the end of this lab, you should be able to:

1. identify the control-signal path and actuator-power path;
2. verify actuator voltage, driver capability, polarity, and common ground;
3. begin an actuator program in a stopped or safe state;
4. command servo position or DC-motor direction and speed;
5. connect a sensor or user input to actuator motion;
6. explain why an Arduino I/O pin must not directly power a motor; and
7. demonstrate a safe sense-decide-act system.

1. Lab Organization

This is one graded lab completed during the shortened Week 5 meeting.

Part	Main work
Common Sections	Component ratings, power-path planning, wiring inspection, and safe startup
Track S	Servo position control and potentiometer-controlled angle
Track D	DC-motor driver, direction control, PWM speed, and sensor/user input
Final Demonstration	Show safe actuator behavior and complete the final reflection

Assigned Track

Complete the common sections and the track assigned by your instructor.

Track S: Servo	**Track D: DC Motor**
☐ assigned	☐ assigned

2. Materials

Item	Purpose
Arduino Uno and USB data cable	Controller, programming, and logic power
Breadboard and jumper wires	Temporary signal and sensor connections
Potentiometer	User input for position or speed command
Computer with Arduino IDE	Code editing, upload, and Serial Monitor
Track S: hobby servo	Limited-angle position actuator
Track D: DC motor and motor driver	Continuous rotation, direction, and PWM speed control
Track D: suitable motor supply	Provides motor operating current
Optional HC-SR04 or push button	Sensor-based or user-based actuator command

Actuator Safety

1. Disconnect USB and external actuator power before major wiring changes.
2. Never power a DC motor directly from an Arduino I/O pin.
3. Verify motor or servo voltage before applying power.
4. Keep fingers, hair, clothing, wires, and loose objects away from moving parts.
5. Begin with the actuator unloaded and with a low-speed or safe-position command.
6. Stop power immediately if a component becomes hot, smells unusual, smokes, or moves unexpectedly.
7. Do not hold or intentionally stall an energized motor or servo.

Lab 5 Common Preparation

Ratings, Power Paths, and Safe Startup

Complete Before Track S or Track D

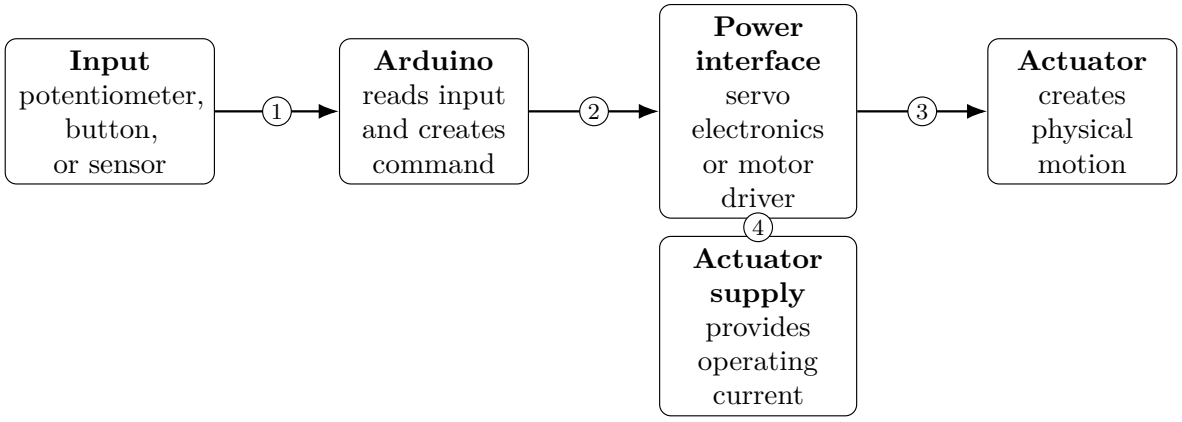
3. Part A: Identify the Assigned Hardware

Record the printed specifications or instructor-provided values.

Component Information

Item	Recorded value
Assigned actuator type	
Actuator model or description	
Recommended actuator voltage	
Motor-driver model, if used	
Driver motor-voltage range, if used	
Motor-supply voltage, if used	
Arduino control pins assigned	

4. Part B: Separate Control from Power



1 measurement **2** control signal **3** switched actuator power **4** actuator-supply input

Answer the following:

1. Which device decides the actuator command?

2. Which source provides most of the actuator operating current?

3. Why is the actuator not normally connected directly to an Arduino I/O pin?

5. Part C: Pre-Power Inspection

Complete this checklist before connecting USB or actuator power.

Check	Verification
☐	Actuator voltage is compatible with the selected supply.
☐	Motor driver is present for the DC-motor track.
☐	Positive and ground connections match the component labels.
☐	Arduino ground and actuator-interface ground share a common reference.
☐	Control-signal pins match the program.
☐	The program begins stopped or at a safe position.
☐	The actuator is unloaded and can move without hitting objects.
☐	Wires cannot enter wheels, gears, horns, or linkages.

Checkpoint 1: Explain Before Power

In two or three sentences, trace the control-signal path and the actuator-power path in your assigned system.

Track S: Servo Position Control

Complete This Track When Assigned

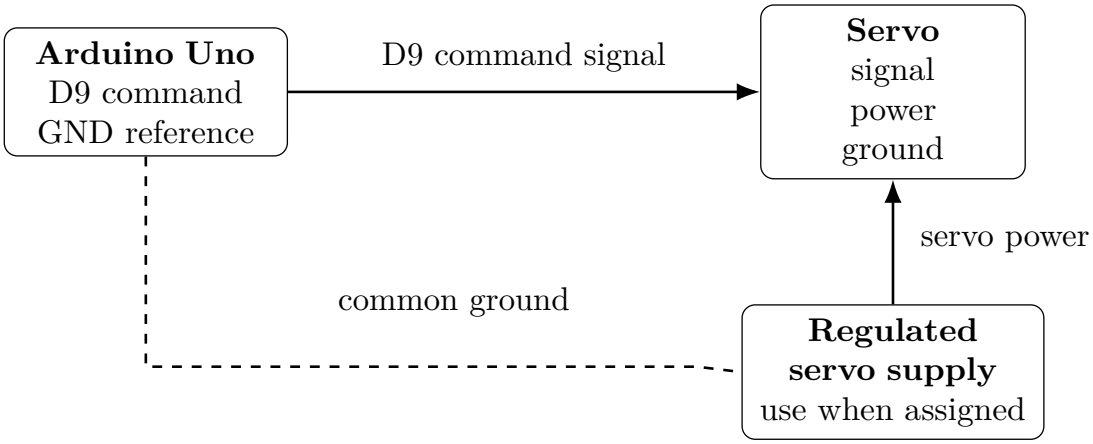
Safe Position Commands and Potentiometer Input

6. Part S1: Wire the Servo

A hobby servo normally has three connections:

- power;
- ground;
- command signal.

Wire colors vary. Use the label, datasheet, or instructor guidance rather than color alone.



Wiring Steps

1. Disconnect USB and any external supply.
2. Connect the servo signal lead to Arduino D9.
3. Connect servo ground to the common ground.
4. Connect servo power to the instructor-approved source.
5. Keep the servo horn clear of wires and objects.
6. Recheck polarity before applying power.

Servo Power

Use only the power arrangement assigned by the instructor. A loaded servo can draw enough current to reset the Arduino or overheat wiring.

7. Part S2: Begin at a Safe Position

Upload this program before attaching a mechanical load.

```
#include <Servo.h>

Servo myServo;

const int servoPin = 9;
const int safeAngle = 90;

void setup() {
  myServo.attach(servoPin);
  myServo.write(safeAngle);
}

void loop() {
  // Hold the safe position.
}
```

Apply power while keeping hands and wires away from the horn.

Initial Test

Observation	Result
Servo moved to approximately 90°	☐ yes ☐ no
Unexpected vibration or repeated motion	☐ yes ☐ no
Arduino reset during movement	☐ yes ☐ no
Component or wire became warm	☐ yes ☐ no

8. Part S3: Test Commanded Angles

Replace the empty `loop()` with:

```
void loop() {
  myServo.write(30);
  delay(1200);

  myServo.write(90);
  delay(1200);

  myServo.write(150);
  delay(1200);
}
```

Do not force the servo horn by hand.

Servo Position Test

Command	Approximate observed angle	Notes
30°		
90°		
150°		

1. Was the observed angle exactly equal to every command? Explain one reason for a difference.

2. Why is a position command different from a measured position?

9. Part S4: Add Potentiometer Control

Wire the potentiometer:

- one outside terminal to `5V`;
- the other outside terminal to `GND`;
- center terminal to A0.

Use this program:

```
#include <Servo.h>

Servo myServo;

const int servoPin = 9;
const int potPin = A0;

void setup() {
  myServo.attach(servoPin);
  myServo.write(90);
  Serial.begin(9600);
}

void loop() {
  int sensorValue = analogRead(potPin);

  int angle = map(sensorValue,
                  0, 1023,
                  20, 160);

  myServo.write(angle);

  Serial.print(sensorValue);
  Serial.print(",");
  Serial.println(angle);

  delay(20);
}
```

The restricted range from 20° to 160° reduces the chance of forcing the servo against a mechanical limit.

Potentiometer-to-Angle Test

Potentiometer position	A0 reading	Angle command	Motion notes
Near minimum			
Near center			
Near maximum			

10. Part S5: Improve the Motion

Rapid changes in the input can create abrupt servo motion. Add a rate limit by moving only one degree per program update.

```
int currentAngle = 90;

void loop() {
  int sensorValue = analogRead(potPin);

  int targetAngle = map(sensorValue,
                        0, 1023,
                        20, 160);

  if (targetAngle > currentAngle) {
    currentAngle++;
  }
  else if (targetAngle < currentAngle) {
    currentAngle--;
  }

  myServo.write(currentAngle);
  delay(15);
}
```

Compare direct motion with rate-limited motion.

Motion Comparison

Behavior	Direct command	Rate-limited command
Response speed		
Smoothness		
Ability to follow fast input changes		

Checkpoint 2S: Servo System

Explain the sense-decide-act sequence for the potentiometer-controlled servo. Identify the sensed variable, the calculation, and the physical output.

11. Track S Design Challenge

Complete one challenge.

Select	**Challenge**	Requirement
☐	**Three-position controller**	A button or potentiometer selects 30°, 90°, or 150°.
☐	**Distance-controlled angle**	Map a valid HC-SR04 distance range to a safe servo angle range.
☐	**Automatic scan**	Sweep between two safe angles and stop at each end without using a long blocking delay.

Describe the completed behavior:

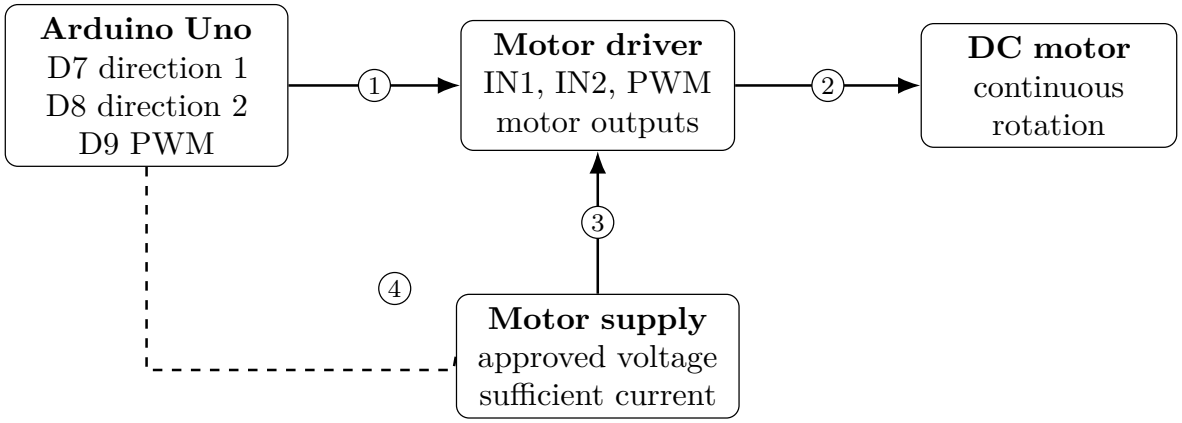
Track D: DC-Motor Driver Control

Complete This Track When Assigned

Direction, PWM Speed, and Safe Reversal

12. Part D1: Wire the Motor Driver

Driver pin names vary. Match the instructor-provided driver diagram.



1 control signals **2** switched motor power **3** motor-supply input **4** common ground

Wiring Steps

1. Disconnect USB and motor power.
2. Connect driver logic inputs to the assigned Arduino pins.
3. Connect the driver outputs to the motor.
4. Connect the approved motor supply to the driver motor-power terminals.
5. Connect Arduino ground and driver logic ground.
6. Place the motor so its shaft, wheel, or gear can rotate freely.
7. Recheck polarity and terminal labels before applying power.

Do Not Connect Motor Power to an I/O Pin

The Arduino supplies command signals. Motor current must flow through the driver and the approved motor supply.

13. Part D2: Begin Stopped

Upload the program with motor power disconnected.

```
const int pwmPin = 9;
const int in1 = 7;
const int in2 = 8;

void setup() {
  pinMode(pwmPin, OUTPUT);
  pinMode(in1, OUTPUT);
  pinMode(in2, OUTPUT);

  digitalWrite(in1, LOW);
  digitalWrite(in2, LOW);
  analogWrite(pwmPin, 0);
}

void loop() {
  // Motor remains stopped.
}
```

Connect motor power after confirming that the motor begins stopped.

14. Part D3: Test Direction Safely

Use the following sequence:

```
void loop() {
  // Direction A
  digitalWrite(in1, HIGH);
  digitalWrite(in2, LOW);
  analogWrite(pwmPin, 120);
  delay(2000);

  // Stop before reversing
  analogWrite(pwmPin, 0);
  delay(700);

  // Direction B
  digitalWrite(in1, LOW);
  digitalWrite(in2, HIGH);
  analogWrite(pwmPin, 120);
  delay(2000);

  // Stop
  analogWrite(pwmPin, 0);
  delay(1200);
}
```

Direction Test

IN1	IN2	PWM	Observed behavior
HIGH	LOW	120	
LOW	HIGH	120	
Either direction	0		

1. Why does the program stop before reversing?

2. Was direction A clockwise or counterclockwise from your viewing position?

15. Part D4: Test PWM Speed

Keep one direction active and test several PWM commands.

```
digitalWrite(in1, HIGH);
digitalWrite(in2, LOW);
```

```
3
4  analogWrite(pwmPin, 80);
5  delay(2500);
6
7  analogWrite(pwmPin, 140);
8  delay(2500);
9
10 analogWrite(pwmPin, 200);
11 delay(2500);
12
13 analogWrite(pwmPin, 0);
```

PWM Test

PWM	Motor started?	Relative speed	Sound, vibration, or other notes
80			
140			
200			

Low PWM Does Not Guarantee Motion

A motor may buzz without rotating when the command is too small to overcome friction and load. Do not leave the motor stalled.

16. Part D5: Potentiometer-Controlled Speed

Wire the potentiometer center terminal to A0 and the outside terminals to `5V` and `GND`.

```
const int potPin = A0;

void loop() {
  int sensorValue = analogRead(potPin);

  int speedValue = map(sensorValue,
                       0, 1023,
                       0, 220);

  digitalWrite(in1, HIGH);
  digitalWrite(in2, LOW);

  if (speedValue < 70) {
    speedValue = 0;
  }

  analogWrite(pwmPin, speedValue);

  Serial.print(sensorValue);
  Serial.print(",");
  Serial.println(speedValue);

  delay(30);
}
```

The deadband below 70 prevents the program from commanding a weak value that may only stall or buzz.

Input-to-Speed Test

Potentiometer position	A0 reading	PWM command	Motor behavior
Near minimum			
Near center			
Near maximum			

17. Part D6: Add Controlled Acceleration

Abrupt PWM changes can create mechanical and electrical shock. Use a ramp:

```
int currentSpeed = 0;

void loop() {
  int sensorValue = analogRead(potPin);

  int targetSpeed = map(sensorValue,
                        0, 1023,
                        0, 220);

```

```
10    if (targetSpeed < 70) {
11      targetSpeed = 0;
12    }
13
14    if (targetSpeed > currentSpeed) {
15      currentSpeed++;
16    }
17    else if (targetSpeed < currentSpeed) {
18      currentSpeed--;
19    }
20
21    digitalWrite(in1, HIGH);
22    digitalWrite(in2, LOW);
23    analogWrite(pwmPin, currentSpeed);
24
25    delay(10);
26  }
```

Direct Command vs. Ramp

Behavior	Direct PWM	Ramped PWM
Startup motion		
Response to a fast input change		
Sound or mechanical shock		

Checkpoint 2D: DC-Motor System

Explain:

1. how the driver reverses motor direction;
2. how PWM changes motor behavior; and
3. why common ground is needed.

18. Track D Design Challenge

Complete one challenge.

Select	**Challenge**	Requirement
☐	**Button direction control**	A button changes direction only after the motor stops briefly.
☐	**Distance safety stop**	Motor runs only when valid distance is greater than a chosen safe threshold.
☐	**Three-zone speed control**	Use safe, warning, and danger distance zones to command fast, slow, and stop.
☐	**Automatic ramp cycle**	Accelerate, hold, decelerate, stop, and reverse without abrupt direction changes.

Describe the completed behavior:

19. Final Demonstration

Demonstrate the assigned track with the following features:

Check	Demonstration requirement
☐	Program begins stopped or at a safe servo position.
☐	Actuator wiring uses the correct power source and common ground.
☐	Actuator responds to a potentiometer, button, or sensor.
☐	Motion remains within a safe speed, direction, or angle range.
☐	Program handles the minimum-input condition safely.
☐	Student can explain the sense-decide-act sequence.
☐	Student can identify the control-signal path and power path.

Checkpoint 3: Safety Explanation

Describe one possible electrical failure and one possible mechanical failure in your system. For each failure, state how the design or procedure reduces the risk.

20. Final Reflection

Answer in complete sentences.

1. Which actuator track did you complete, and what physical motion did the actuator create?

2. Trace the full sense-decide-act sequence in your final system.

3. What is the difference between the Arduino command signal and the actuator operating power?

4. What evidence showed that the input successfully controlled the actuator?

5. Identify one limitation of your system. Consider measurement noise, load, speed, position accuracy, available current, or response time.

6. Describe one improvement that would make the system safer or more reliable.

21. Submission Checklist

Submit one worksheet per student unless your instructor specifies otherwise.

Check	Required item
☐	Name, partner, date, and assigned track are recorded.
☐	Component specifications and control pins are recorded.
☐	Common preparation questions are complete.
☐	All data tables for the assigned track are complete.
☐	Checkpoint explanations are complete.
☐	One design challenge is demonstrated and described.
☐	Final reflection is complete.
☐	Final Arduino code is submitted as instructed.

Power Down Before Leaving

Set the actuator command to stop or a safe position. Disconnect motor or servo power, disconnect USB, and return all hardware as instructed.